ZETTA TECH

INTERNSHIP ASSESSMENT

Answering-Machine Detection

Decide, from the first seconds of a live phone call, whether a person picked up or a machine did — and decide it fast enough to matter.

DURATION	SUBMISSION	METRIC
2 hours + 30 min write-up	**GitHub repository**	**Detection latency @ 5% FH**

The problem

Zetta runs an outbound dialer. When a call connects, something on the other end starts making noise — and the dialer has to decide, fast, which of these it is:

- a **human** who picked up ("Hello?", "Haan, boliye", "Yes who is this")
- a **machine** — voicemail greeting, IVR menu, or a phone's call-screening assistant

Get it wrong in one direction and you hand a live agent a voicemail: the agent burns 30 seconds listening to a recording. Get it wrong in the other direction and you **hang up on a real customer**, which is far worse.

And it has to be quick. Every extra second you spend deciding is a second of silence on a real human's ear.

Your task: build a model that makes this decision from the audio, as early as it responsibly can.

The data (`amd_data/handout/`)

```
amd_data/handout/
  audio/          amd_0001.wav ...   (8 kHz mono, one call opening per file,
                                      starting the moment the call connects)
  labels.csv
```

COLUMN	MEANING
`call_id`	matches the wav filename
`audio_file`	path relative to the data dir
`label`	`human` = a person answered · `machine` = voicemail / IVR / screener
`subtype`	`voicemail` · `screener` · `human` — metadata only, you will not get this at test time

221 calls, 49 of them machines. The audio is real production telephony: narrowband, μ-law, and in places genuinely damaged — packet loss, silence suppression, AGC pumping. That is what the line actually sounds like. It is not noise to filter out and forget; some of it is the problem.

Two properties we set deliberately, so you do not waste time on them:

- **Every clip is exactly 8.00 seconds.** Length tells you nothing.
- **Call ids are random.** They encode nothing about the label or the ordering.

The corpus is predominantly English. The answering systems in it are carrier voicemail and phone call-screening assistants.

One warning we would rather give you than spring on you. Not every machine speaks. Some answered calls are a beep, a tone, hold music, or near-silence with no greeting at all. There are none of

those in your handout, and there are some in the hidden set. If your model can only recognise a machine by hearing it talk, it will miss them completely — and a miss costs you the full 6000 ms.

Your task

Write a model that, for each call, outputs `p_machine` at a series of **checkpoints** through the call: how confident are you, *by this moment*, that this is a machine.

Checkpoints are every 250 ms from 500 ms to 6000 ms.

Causality rule (hard requirement). For the checkpoint at time *T*, your features may use audio **only from 0 to T**. Never audio after *T*. A live dialer cannot hear the future. We read your feature code to check this, and we will ask you about it.

Deliverables (all five)

1. `predict.py` — runs as `python predict.py --data_dir <folder> --out predictions.csv` It must work on a folder with the same structure it has never seen. `predictions.csv` columns: `call_id,checkpoint_ms,p_machine`.
2. `predictions.csv` for the handout set.
3. `RUNLOG.md` — after every scoring run: the score, and one or two lines on what you changed and why. **This is graded, and it is worth a quarter of your mark.** We are reading for whether each step followed from the last one. - **Commit it as you go, not at the end.** We check the git history: a run log that first appears in your final commit is a write-up, not a log, and it is marked as one. - **Include at least two things you tried that did NOT work, with the numbers.** A dead end you measured and abandoned tells us more than a success you stumbled into. A log with no failures in it is not a real log.
4. `NOTES.md` — max 10 sentences: what signal your model actually uses, where it still fails, what you would do with one more day. Must also include: - **which AI coding tool and model you used** (e.g. "Claude Code, Sonnet, free tier" / "Cursor, GPT-5" / "no assistant"). Declare it honestly; it does not count against you, see Grading. - **any prior art you found and what you took or rejected from it.**
5. `SUMMARY.html` — a single self-contained page, built with your coding agent, covering your approach, your results with a graph or two, and a plain summary of the two MD files. It must also state, explicitly: - **what you did and what the coding agent did.** Be specific. "The agent wrote the feature extraction, I found the VAD was firing on packet loss and rewrote the floor estimate" is the kind of sentence we are looking for. - **why your solution beats the reference numbers**, in your own words. - **the two or three prompts that changed the direction of your work, verbatim.** Not your whole session — the ones where you told the agent something it did not already know, or redirected it after it went wrong. Copy them in as you wrote them, typos and all. This is the clearest evidence of what you contributed, and it is the same for every tool, so nobody is advantaged by which assistant they can afford.

No deployment needed; a local HTML file is fine.

Submitting

Push to a **private GitHub repository** and give us the link on the submission form.

Add `kamal-zetta` as a collaborator. Settings → Collaborators → Add people → `kamal-zetta`. That is how we read your work.

Keep the repository private. Do not publish it — your solution is yours, and a public repo also means next year's candidates can find it.

A repository we cannot open cannot be graded, and forgetting the invite is the single most common way people lose an assignment they actually did well on. Send it before you submit the form, not after.

Also:

- `predict.py` **in the repository root** (we also look in `starter/` and `src/`, and nowhere else).
- **Commit as you go.** We look at the history — a single commit at the end tells us your run log was written afterwards.
- **No absolute paths.** Your code runs on our machine, against a folder it has never seen.
- **Commit everything your `predict.py` imports** — including `starter/features.py` if you use it. We clone your repository and run it on its own; anything you left out because "it came with the kit" will not be there, and the run dies with `ModuleNotFoundError`. The safest check: copy your repo to an empty folder and run `predict.py` there.
- Run `python selfcheck.py` last. If it passes, we can grade you.

Scoring

```
python score.py --data_dir amd_data/handout --pred predictions.csv
```

The scorer replays your probabilities as if it were a live dialer: it hangs up the instant `p_machine` crosses a threshold. It tunes that threshold for you, picking the **fastest** operating point that still keeps the false-hangup rate within budget.

> **Headline metric: mean detection latency (ms) on machine calls, at a false-hangup rate ≤ 5%. Lower is better.**

A machine you never detect is scored at the 6000 ms deadline, so misses cost you latency. Two things therefore do not work: always saying "machine" (you hang up on humans), and never saying it (you eat the deadline).

Reference points on this exact metric:

SYSTEM	LATENCY @ 5% FALSE-HANGUP
never firing at all (the floor)	6000 ms
silence/duration heuristic (`starter/baseline.py` , given to you)	**fails — no operating point stays inside the budget**
memorising the handout greetings (nearest-neighbour)	**6000 ms — it fails completely**
our own first attempt, acoustic features + gradient boosting	~3100 ms
a coding agent given this kit and no human insight	**~2470 ms ← this is the bar**

Read that table carefully — every number in it we measured on the real hidden set.

The heuristic we hand you does not work. Neither does any rule built only on how long someone has been talking: we tried several and none separates a person from a recording at this false-hangup budget. People on these calls talk plenty. Timing alone is a dead end, and finding that out for yourself in the first fifteen minutes is time well spent.

Memorising does not work either. We tested that too — a nearest-neighbour matcher that recognises the exact greetings in your handout scores the 6000 ms floor, because the hidden set's greetings are not in your handout. See the note under Grading.

Note the last two rows, because they are the honest shape of this problem: **an unassisted coding agent beat our own first attempt.** That is not a joke at our expense, it is the point. Driving an agent well gets you to the bar. Getting meaningfully below it takes something the agent does not have — you looking at the audio it got wrong and working out why.

That gap is the assignment.

Run `python selfcheck.py` before you submit. It runs your `predict.py` against **eight real calls that are not in your handout**, checks your output schema, and prints your score on those eight. It is a smoke test, not a score to trust or tune against — eight calls tells you almost nothing. If selfcheck fails, your submission will fail on our machine too.

What we are grading

Not only the number.

WEIGHT	WHAT
30%	Score, measured **above the coding-agent baseline**
25%	`RUNLOG.md` — did your iterations build on each other, or did you thrash?
20%	Judgement — what you noticed about the data and the metric that wasn't handed to you
15%	Code quality; runs unmodified on unseen data
10%	`NOTES.md` honesty about your own failure modes

On AI coding assistants: use them. They are how we work here. We run this kit with a coding agent and no human insight, and we know what it reaches on its own. **Your grade lives in the gap above that number.** If you hand the problem over and accept the first answer, you will land near it, and we will see that in your run log. A candidate on a weaker free-tier model who adds real insight scores higher than one on the best model who adds none — which is exactly why we ask you to declare your tooling, and why lying about it is the one thing that ends your application.

The final grade uses a **hidden test set of real production calls from our dialer**, on our machine, plus your run log and a **5-minute discussion of your model**.

Two things about that hidden set you should know before you start, because they change what is worth building:

1. **It contains answering-machine systems that are not in your training data.** Different carriers, different greetings, different voices. A model that learns to recognise *the specific recordings in the handout* will score badly. A model that learns *what makes a recording a recording* will not.
2. **The class balance is different**, and there are more machines in it than the handout might lead you to expect.

So: features that generalise beat features that memorise. That is the whole game here.

Rules

- Laptop CPU only. No GPUs, no cloud training.
- Allowed: exactly what is in `requirements.txt` — `numpy`, `scipy`, `scikit-learn`, `pandas`, `librosa`, `soundfile`. That is the interpreter we grade in, so anything else will not import on our machine.
- **Not allowed:** any pretrained model or downloaded weights of any kind — no Whisper, wav2vec, Silero, webrtcvad, HuggingFace models, no ASR/TTS APIs. No external datasets. Algorithms are fine (a pitch tracker is an algorithm; a pretrained VAD is a model).
- Everything offline. The kit contains everything you need.
- **Save your trained model with `numpy.savez` or JSON, not pickle.** We grade forty submissions in a sandbox and we do not unpickle anything, so a pickle-only model cannot be scored. `starter/predict.py` shows the pattern.

- AI coding assistants allowed and encouraged. Declare them in `NOTES.md`.

Suggested path (not mandatory)

- **0–15 min** — run `starter/baseline.py`, score it, then **listen to five wavs where the baseline gets it wrong**. Understand why duration alone fails.
- **15–60 min** — build features over the audio available at each checkpoint (`starter/features.py` has loading, a simple VAD and a pitch tracker). Train a small classifier. Score it.
- **60–100 min** — look at your worst errors, by subtype. Fix what you see. Iterate, and log each run.
- **100–120 min** — finalise `predictions.csv`, run `selfcheck.py`.
- **+30 min** — `SUMMARY.html`, `RUNLOG.md`, `NOTES.md`.

Good luck.